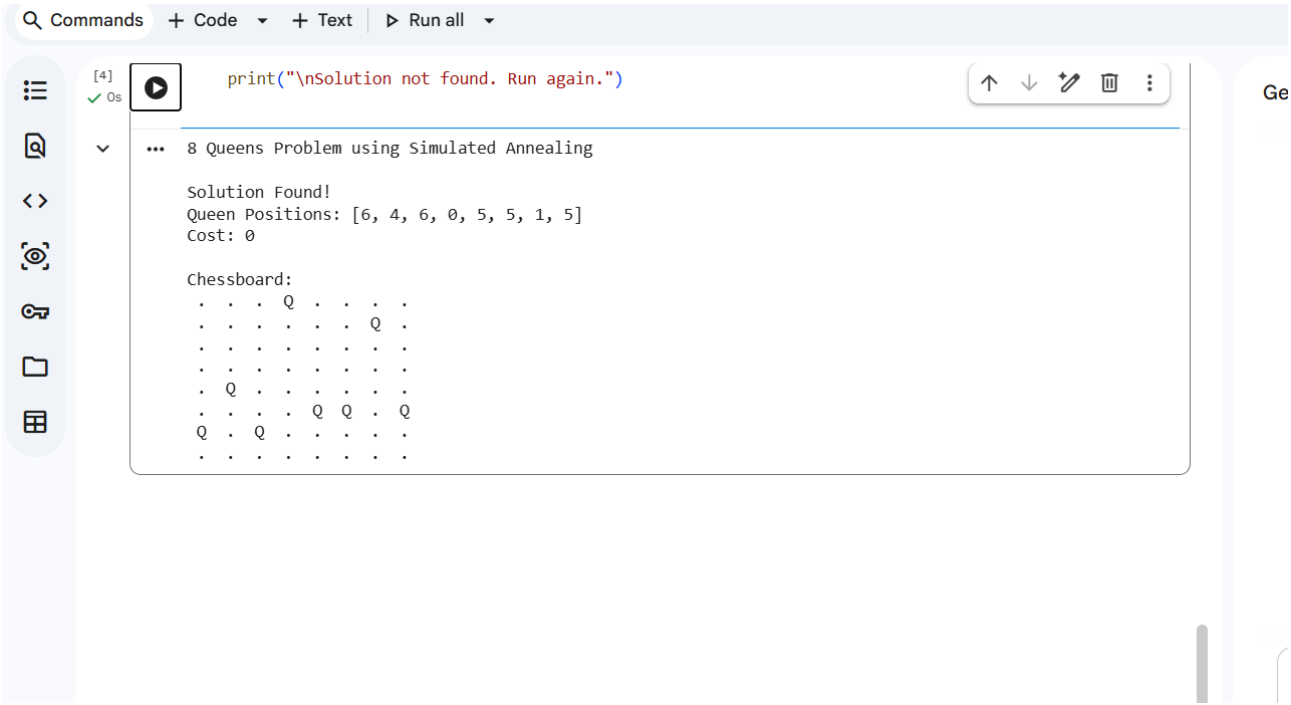
Code

```
import random
import math

def cost(state):
    conflicts = 0

    for i in range(8):
        for j in range(i + 1, 8):
            if abs(state[i] - state[j]) == abs(i - j):
                conflicts += 1

    return conflicts


def random_neighbour(state):
    next_state = state.copy()

    col = random.randint(0, 7)
    row = random.randint(0, 7)

    while row == next_state[col]:
        row = random.randint(0, 7)

    next_state[col] = row

    return next_state


def simulated_annealing():

    current = [random.randint(0, 7) for _ in range(8)]

    T = 100.0
    cooling_rate = 0.99

    while T > 0.001:

        next_state = random_neighbour(current)

        delta_E = cost(current) - cost(next_state)

        if delta_E > 0:
            current = next_state

        else:
            p = math.exp(delta_E / T)

            if random.random() < p:
                current = next_state

        if cost(current) == 0:
            return current

        T = T * cooling_rate

    return current if cost(current) == 0 else None


def display_board(state):

    for row in range(8):
        for col in range(8):

            if state[col] == row:
                print(" Q ", end="")
            else:
                print(" . ", end="")

        print()


print("8 Queens Problem using Simulated Annealing")

solution = simulated_annealing()

if solution is not None:
    print("\nSolution Found!")
    print("Queen Positions:", solution)
    print("Cost:", cost(solution))
    print("\nChessboard:")
    display_board(solution)

else:
    print("\nSolution not found. Run again.")
```

Output


Commands
+ Code
+ Text
Run all
[4]
0s
print("\nSolution not found. Run again.")
8 Queens Problem using Simulated Annealing
Solution Found!
Queen Positions: [6, 4, 6, 0, 5, 5, 1, 5]
Cost: 0
Chessboard:
. . . Q . . . .
. . . . . . Q .
. . . .
. . . .
. Q . . . . . .
. . . . Q Q . Q
Q . Q . . . . .
. . . .
Ge